Peer-Review 1: UML

Sara Piazza, Theodora Andrada Pascu, Francesco Pesce, Davide Pepe

Gruppo GC50

Valutazione del diagramma UML delle classi del gruppo GC40.

Lati positivi

L'architettura ci è sembrata nel complesso compatta e la maggior parte delle classi e dei metodi ha un nome autoesplicativo.

Abbiamo apprezzato il fatto che abbiano pensato anche ad alcune funzionalità avanzate, in particolare la gestione di partite multiple in parallelo e la resilienza alle disconnessioni.

Ci è sembrata efficace la decisione di gestire i common goal con uno strategy pattern e di raggruppare quelli simili fra di loro per non ripetere codice e per non appesantire inutilmente il codice e l'UML.

Lati negativi

Lo schema in alcuni punti era poco chiaro per via della sovrapposizione di alcune frecce e per la disposizione poco efficace delle classi.

La creazione della classe astratta Card non porta vantaggi evidenti dato che contiene solo metodi astratti (non permettendo di conseguenza il riutilizzo di codice) e inoltre nessun'altra classe ha attributi di tipo Card utilizzando sempre CommonGoalCard o PersonalGoalCard.

Non ci è sembrato utile dividere il concetto di CommonGoalCard da quello di Goal, riteniamo che sia più semplice unire le due classi in una sola da cui potrebbero ereditare le varie classi che implementano l'interfaccia Goal.

Non è molto chiaro il modo in cui sono stati numerati i common goal e di conseguenza il modo in cui sono stati raggruppati. In nessun punto delle regole del gioco viene fatto riferimento ai common goal attraverso un numero e inoltre l'ordinamento delle common goal card nelle regole in italiano è diverso da quelle in inglese. Abbiamo provato a dare varie interpretazioni alla numerazione ma nessuna sembrava avere senso con i raggruppamenti che erano stati fatti. Per lo stesso motivo non risulta chiara la funzione del metodo updateRC in G1_G3.

Non viene tenuta traccia di quali token hanno raccolto i player ma solamente della somma dei loro punteggi e di quali common goal sono stati completati, di conseguenza in alcuni casi può essere impossibile risalire a quali token il player ha raccolto per poterli mostrare a schermo.

In alcune classi ci sono degli attributi e dei metodi indicati con l’iniziale maiuscola che per convenzione viene adottata solo dalle classi.

Confronto tra le architetture

Un punto comune fra le architetture è la scelta di realizzare i common goal utilizzando un pattern strategy e di raggrupparne alcuni con lo scopo di riutilizzare parti di codice.

L’architettura che stiamo valutando gestisce anche alcune funzionalità avanzate mentre noi abbiamo deciso di occuparcene più avanti.

La nostra living room board è una lista di TileSpace che contengono una Tile mentre quella del gruppo GC40 è una matrice di ItemTile. Il vantaggio della nostra scelta è che non dobbiamo creare degli ItemTile di tipo NOTREACHABLE per tenere traccia del fatto che la living room board non sia un rettangolo, in questo modo funziona bene per qualsiasi forma di living room board senza occupare memoria non necessaria. Il vantaggio principale della loro scelta invece è l’accesso diretto al singolo ItemTile tramite le coordinate.

Un’ulteriore differenza è il fatto che noi abbiamo scelto di implementare una classe Deck che gestisce l’estrazione di un oggetto causale e tre classi (Bag, CommonGoalDeck, PersonalGoalDeck) che ereditano da essa e si occupano dell’inizializzazione dei mazzi. Il gruppo valutato invece tiene traccia dei mazzi di carte all’interno di Match, il che rende la classe Match più complessa ma diminuisce il numero di classi totali.